Trabajo Práctico – Clases Abstractas, Derivadas, Gestoras e Interfaces

Ejercicio 1: Diseño de las Clases de Entidad Básicas

Crear las siguientes clases con sus propiedades automáticas y constructores. Cada clase debe incluir un método `MostrarDatos()` para imprimir su información.

Clase `ProductoMenu`:

- Propiedades:
  - `int IdProducto { get; private set; }`
  - `string Nombre { get; set; }`
  - `string Descripcion { get; set; }`
  - `decimal Precio { get; set; }`
  - `bool Disponible { get; set; }` (inicializar en `true` por defecto)
- Constructor: que reciba `idProducto`, `nombre`, `descripcion`, `precio`.
- Método: `void MostrarDatos()` que imprima todos los detalles del producto.

Clase `ItemPedido`:

- Propiedades:
  - `ProductoMenu Producto { get; private set; }`
  - `int Cantidad { get; set; }`
- Constructor: que reciba un objeto `ProductoMenu` y la `cantidad`.
- Método: `void MostrarDatos()` que imprima el nombre del producto, cantidad, precio unitario y subtotal (`Cantidad * Producto.Precio`).

Ejercicio 2: Creación de la Primera Clase Gestora: `GestorMenu`

Crear la clase GestorMenu, que será responsable de administrar la colección de objetos ProductoMenu disponibles en el restaurante.

Requisitos:

- La clase debe poseer una colección interna (por ejemplo, List<ProductoMenu>) para almacenar todos los productos del menú.

Métodos a implementar:

- bool AgregarProducto(ProductoMenu producto): Agrega un ProductoMenu a la colección, verificando que no haya IdProducto duplicados. Retorna true si se agregó correctamente, false si ya existía.
- void ListarProductos(): Recorre la colección e invoca el método MostrarDatos() de cada ProductoMenu.
- ProductoMenu BuscarProductoPorId(int idProducto): Busca y retorna un ProductoMenu por su IdProducto. Retorna null si no lo encuentra.
- bool ActualizarDisponibilidad(int idProducto, bool disponible): Busca un producto por ID y actualiza su estado Disponible. Retorna true si se actualizó correctamente, false si no se encontró.

Ejercicio 3: Diseño de la Clase `Pedido` y su Clase Gestora: `GestorPedidos`

Para manejar la lógica de los pedidos de los clientes, crearemos la entidad `Pedido` y su respectiva clase gestora `GestorPedidos`.

Clase `Pedido`:

- Propiedades:
  - `int IdPedido { get; private set; }`
  - `int IdCliente { get; set; }`
  - `List<ItemPedido> Items { get; private set; }`
  - `DateTime FechaPedido { get; set; }`
  - `string Estado { get; set; }` (por defecto: `"Pendiente"`)
- Constructor: Recibe `idPedido`, `idCliente`, `fechaPedido`. Inicializa la lista de ítems y el estado.
- Método:
  - `decimal CalcularTotal()`: Recorre la colección `Items` y calcula el total del pedido sumando los subtotales de cada ítem (`Cantidad * Producto.Precio`).

Clase `GestorPedidos`:

- Propiedades:
  - Una colección interna (`List<Pedido>`) para almacenar los pedidos.
  - Un contador interno para generar IDs únicos de pedido.
- Métodos a implementar:

1. `Pedido CrearNuevoPedido(GestorMenu gestorMenu, GestorClientes gestorClientes)`
   - Solicita al usuario el ID del cliente y lo valida usando `gestorClientes`.
   - Permite ingresar múltiples productos (ID y cantidad).
   - Por cada producto válido y disponible, busca el objeto `ProductoMenu` y crea un `ItemPedido` con ese objeto.
   - Si no se agregan ítems válidos, retorna `null`.
   - Si el pedido es válido, lo agrega a la colección y lo retorna.
2. `void ListarPedidos()`
   - Recorre la colección de pedidos e imprime los datos de cada uno, incluyendo:
     - ID del pedido, ID del cliente, fecha, estado.
     - Detalle de cada ítem (nombre del producto, cantidad, precio, subtotal).
     - Total del pedido.
3. `Pedido BuscarPedidoPorId(int idPedido)`
   - Retorna el pedido con el ID indicado, o `null` si no existe.
4. `bool ModificarEstadoPedido(int idPedido, string nuevoEstado)`
   - Valida que el estado sea uno de los permitidos: `"Pendiente"`, `"Preparado"`, `"Entregado"`, `"Cancelado"`.
   - Si el pedido existe y el estado es válido, lo actualiza y retorna `true`. En caso contrario, retorna `false`.

Ejercicio 4: Métodos Adicionales en GestorPedidos y Consultas

Métodos a implementar en GestorPedidos:

1. Pedido BuscarPedidoPorId(int idPedido)
   - Busca y retorna un objeto Pedido de la colección por su IdPedido.
   - Si no lo encuentra, retorna null.
2. bool ModificarEstadoPedido(int idPedido, string nuevoEstado)
   - Busca un pedido por su IdPedido.
   - Valida que el nuevoEstado sea uno de los siguientes: "Pendiente", "Preparado", "Entregado", "Cancelado".
   - Si el pedido existe y el estado es válido, actualiza la propiedad Estado y retorna true.
   - En caso contrario, retorna false.
3. void ListarPedidos()
   - Recorre la colección de pedidos e imprime los datos completos de cada uno:
     - ID del pedido, ID del cliente, fecha, estado.
     - Detalle de cada ítem (Producto.Nombre, cantidad, precio unitario, subtotal).
     - Total del pedido (calculado con Pedido.CalcularTotal()).

Ejercicio 5: Incorporación de Clientes e Integracion con Pedidos

Integra la gestión de clientes en el sistema:

Clase `Cliente`:

- Propiedades:
  - `int IdCliente { get; private set; }`
  - `string Nombre { get; set; }`
  - `string Apellido { get; set; }`
- Constructor: que reciba `idCliente`, `nombre`, `apellido`.

Clase `GestorClientes`:

- Propiedades:
  - Una colección interna (`List<Cliente>`) para almacenar los clientes.
- Métodos a implementar:

  1. `bool AgregarCliente(Cliente cliente)`
     - Agrega un cliente si no existe otro con el mismo ID.
     - Retorna `true` si se agregó correctamente, `false` si ya existía.
  2. `Cliente BuscarClientePorId(int idCliente)`
     - Retorna el cliente con el ID indicado, o `null` si no existe.
  3. `void ListarClientes()`
     - Imprime los datos de todos los clientes registrados.

Ejercicio 6: Interfaz de Consola (Simulación de Vista Principal)

Integra todas las clases y gestoras en el método `Main` para una interfaz de usuario básica por consola.

Requisitos:

1. En el método `Main` de la clase `Program`, crear instancias de:
   - `GestorMenu`
   - `GestorClientes`
   - `GestorPedidos`
2. Implementar un menú interactivo usando un bucle `while` y una estructura `switch` que permita al usuario seleccionar entre las siguientes opciones:

```
1. Agregar Producto al Menú
2. Listar Productos del Menú
3. Actualizar Disponibilidad de Producto
4. Agregar Cliente
5. Listar Clientes
6. Crear Nuevo Pedido
7. Listar Todos los Pedidos
8. Modificar Estado de Pedido
0. Salir
```

Consideraciones:

- Todas las operaciones de búsqueda, validación y visualización deben ser realizadas por las gestoras.
- Las entidades (ProductoMenu, Cliente, Pedido, ItemPedido) no deben contener lógica de presentación ni depender de otras clases para funcionar.
- El método Main actúa como controlador de flujo, delegando las tareas a las gestoras.